Do Ed25519 ao Falcon-512: Viabilidade de uma Transição Pós-Quântica na Solana via Syscall Nativo

Arthur T. Oliveira , Ana Cristina dos Santos , Bryan Ferreira

[1]Instituto de Tecnologia e Liderança

arthur.oliveira@sou.inteli.edu.br,

anacris.santos@prof.inteli.edu.br, bryan.ferreira@prof.inteli.edu.br

Abstract. *The emergence of cryptographically relevant quantum computers poses an existential threat to the cryptographic infrastructure underlying blockchain networks. Solana, a high performance blockchain processing thousands of transactions per second, relies on the Ed25519 digital signature algorithm, which is vulnerable to Shor algorithm. This work investigates the technical feasibility of transitioning Solana to post-quantum cryptography through two complementary approaches: (1) the implementation of a hybrid program combining Ed25519 authentication with Falcon 512 verification using a two layer architecture (off-chain for PQC key management and on-chain for signature verification), and (2) a proposal for modifying the Solana runtime through a native syscall for post-quantum signature verification. The results demonstrate that while executing PQC verificationn directly within the Solana Virtual Machine (SVM) is severely limited by the runtime environmental constraints, shifting this computational workload to the validator runtime via a native syscall bypasses these bottlenecks, making the transition to Falcon512 technically and operationally feasible.*

1. Introdução

O advento de computadores quânticos criptograficamente relevantes (CRQCs) representa uma ameaça existencial à infraestrutura de segurança digital. Conforme demonstrado pelo algoritmo de Shor, a quebra de sistemas baseados em curvas elípticas e fatoração de inteiros em tempo polinomial [1] viabiliza ataques do tipo "Harvest Now, Decrypt Later" [2], comprometendo algoritmos clássicos como RSA e ECDSA. Para sistemas de alta criticidade e imutáveis como as blockchains, cuja segurança de longo prazo depende fundamentalmente dessas primitivas criptográficas [3], essa vulnerabilidade é permanente: todo o histórico de transações já registrado torna-se um alvo perpétuo para ataques futuros.

A rede Solana fundamenta sua segurança no algoritmo de assinatura digital Ed25519 [4] [5], uma variante do EdDSA baseada na curva elíptica Curve25519. O Ed25519 oferece tamanhos compactos de chave pública (32 bytes) e de assinatura (64 bytes), características essenciais para uma rede de alta performance que processa milhares de transações por segundo [6]. A dependência do Ed25519 permeia toda a arquitetura da Solana: o Proof of History utiliza-o para autenticação de registros temporais, o Tower BFT depende de votos assinados pelos validadores para garantir o consenso, e cada transação submetida à rede requer pelo menos uma assinatura Ed25519

válida. Um computador quântico capaz de resolver o problema do logaritmo discreto em curvas elípticas (ECDLP) seria suficiente para comprometer simultaneamente todos esses componentes [1]. Diante dessa ameaça sistêmica à infraestrutura da rede, este trabalho levanta o seguinte questionamento se é tecnicamente viável verificar assinaturas pós-quânticas na Solana mantendo compatibilidade com as restrições rigorosas do seu ambiente de execução (*runtime*).

Como resposta a esse cenário, o National Institute of Standards and Technology (NIST) publicou, em 2024, os primeiros padrões de Criptografia Pós-Quântica: FIPS 203 (ML-KEM para encapsulamento de chaves), FIPS 204 (ML-DSA para assinaturas digitais) e FIPS 205 (SLH-DSA baseado no SPHINCS+) [7]. O NIST recomenda explicitamente uma abordagem de criptografia híbrida durante o período de transição, combinando algoritmos clássicos com algoritmos pós-quânticos para oferecer segurança contra ameaças tanto clássicas quanto quânticas [7]. Entre os algoritmos baseados em reticulados, o Falcon-512 destaca-se por apresentar o menor tamanho de assinatura com, aproximadamente, 666 bytes e chaves públicas de 897 bytes [8] tornando-o o candidato mais adequado para redes com restrições severas de tamanho de transação como a Solana, cujo limite é de 1232 bytes [9].

Contudo, a adoção do Falcon-512 na Solana enfrenta um obstáculo técnico fundamental. A Solana Virtual Machine (SVM) executa programas compilados para bytecode eBPF (Extended Berkeley Packet Filter), um ambiente restrito projetado originalmente para o kernel do Linux. Em arquiteturas de sistema operacional, operações de ponto flutuante são evitadas no espaço do kernel devido ao alto custo computacional de salvar e restaurar os registradores das unidades de pontos flutuantes a cada troca de contexto [12]. Por herdar essa arquitetura, o eBPF da SVM não suporta operações nativas de ponto flutuante. Consequentemnte, a emulação via software dessas operações matemáticas, que são intrínsecas ao Falcon512, excedem drasticamente os limites de unidades computacionais. Essa limitação estrutural torna a verificação direta de assinaturas pós quânticas dentro do program on chain inviável, evidenciando que a adoção do algoritmo de verificação só se torna possível através da implementação de um suporte nativo diretamente no runtime da Solana.

Dessa forma, este trabalho investiga a viabilidade técnica da transição híbrida da rede Solana para criptografia pós-quântica através de duas abordagens complementares: (1) a implementação de um programa híbrido que combina autenticação Ed25519 com armazenamento de chaves Falcon-512 em uma arquitetura de duas camadas (off-chain e on-chain), demonstrando experimentalmente a limitação da SVM; e (2) a proposta de modificação do runtime da Solana através da adição de um syscall nativo (`sol_falcon512_verify`) para verificação de assinaturas pós-quânticas, contornando as restrições do ambiente eBPF ao executar a verificação diretamente no processo do validador.

2. Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho, organizados em dois blocos: a arquitetura da rede Solana, com ênfase nos componentes relevantes para o processo de transição criptográfica, e os fundamentos da criptografia pós-quântica, com foco nos algoritmos baseados em reticulados.

2.1. Arquitetura da Rede Solana

2.1.1. Modelo de Contas

A Solana adota um modelo de dados baseado em contas: todo estado persistente na rede é armazenado em contas, estruturas identificadas por um endereço de 32 bytes (chave pública Ed25519) e gerenciadas por programaas específicos definidos no campo *owner* que indica qual programa tem permissão de escrita sobre seus dados. Esse modelo é fundamental para a arquitetura deste trabalho, pois permite armazenar a chave pública Falcon512 do usuário diretamente on-chain em uma conta controlada pelo programa verificador.

2.1.2. Program Derived Addresses (PDAs)

PDAs são endereços determinísticos derivados de um *program ID* e *seeds* definidas pelo desenvolvedor [21]. Diferentemente de contas regulares, PDAs são projetados para existir fora da curva Ed25519 através de um parâmetro de ajuste (*bump*), o que garante que não possuam uma chave privada correspondente:

$$PDA = SHA256(seed_1 \| seed_2 \| \ldots \| seed_n \| program_id \| bump) \tag{1}$$

Como consequência, apenas o programa proprietário tem autoridade para assinar instruções e modificar os dados dessa conta. Na arquitetura proposta, essa propriedade é fundamental, já que o PDA vincula de forma segura a identidade Ed25519 do usuário à sua chave pública Falcon-512, permitindo que o programa gerencie o estado e verifique a autenticidade pós-quântica de forma autônoma on-chain.

2.1.3. Solana Virtual Machine (SVM)

A SVM é o ambiente de execução on-chain da Solana, baseado no extended Berkeley Packet Filter (eBPF). Além da restrições significativas referente às operações de pontos flutuantes, variáveis globais mutáveis ou passagens de estruturas complexas impostas pelo ambiente eBPF [12], a SVM impõe limites de Compute Units (CUs): 200.000 CUs por instrução e 1.400.000 CUs por transação. Essas restrições são o obstáculo central para a verificação Falcon-512 on-chain, uma vez que o algoritmo depende de aritmética de ponto flutuante. [20].

2.1.4. Syscalls Nativos

As syscalls são funções implementadas diretamente no runtime da Solana, executadas fora do ambiente eBPF com acesso aos recursos do sistema hospedeiro. Conceitualmente, seguem o mesmo princípio das chamadas de sistema em sistemas operacionais tradicionais, em que um processo em modo usuário solicita serviços ao kernel através de uma interface controlada [19]. Na Solana, exemplos incluem `sol_sha256` para hashing

e `sol_ed25519_verify` para verificação de assinaturas Ed25519. As syscalls consomem um número fixo de CUs independentemente dos dados de entrada, oferecendo previsibilidade de custo. Dessa forma, syscalls permitem deslocar a verificação criptográfica para o runtime da rede, evitando as limitações computacionais impostas pelo ambiente eBPF.

2.2. Criptografia Pós-Quântica

2.2.1. Vulnerabilidade de Algoritmos Clássicos

Os esquemas de assinatura digital baseados em curvas elípticas, como o Ed25519, fundamentam sua segurança na dificuldade computacional do Problema do Logaritmo Discreto em Curvas Elípticas (ECDLP). Em termos formais, dado um ponto gerador G em uma curva elíptica e um ponto público $Q = kG$, a segurança do esquema depende da dificuldade de determinar o escalar k a partir de G e Q [14]. No modelo computacional clássico, os melhores algoritmos conhecidos para resolver o ECDLP possuem complexidade exponencial, o que confere ao Ed25519 um nível de segurança equivalente a 128 bits [4].

Contudo, o algoritmo de Shor demonstra que um computador quântico com número suficiente de qubits pode resolver tanto o problema da fatoração de inteiros, quanto o ECDLP em tempo polinomial [1]. Especificamente, para curvas elípticas sobre corpos finitos $\mathbb{F}_p$, o algoritmo de Shor requer $\mathcal{O}(\log^3 p)$ operações quânticas, reduzindo a complexidade de exponencial para polinomial e eliminando a segurança proporcionada pelo ECDLP [10] [11]. Isso significa que a segurança de 128 bits do Ed25519 seria completamente comprometida por um computador quântico criptograficamente relevante.

2.2.2. Escolha do algoritmo PQC: Falcon-512

A seleção do algoritmo pós-quântico para a Solana é condicionada por uma restrição arquitetural fundamental: o tamanho máximo de uma transação na rede é de **1.232 bytes**. Como a assinatura PQC e eventuais metadados devem caber dentro desse limite junto com os dados da instrução, algoritmos com assinaturas ou chaves públicas muito grandes tornam-se inviáveis sem fragmentação de transações. A Tabela 1 apresenta os principais algoritmos padronizados pelo NIST [7] e sua compatibilidade com esse limite.

Algoritmo	PK (bytes)	Sig (bytes)	PK + Sig	Compatível?
Ed25519 (atual)	32	64	96	Sim (7,8%)
Falcon-512	897	666	1.563	Não (PK + Sig)
Falcon-512 (PK em PDA)	—	666	666	**Sim (54%)**
Falcon-1024	1.793	1.280	3.073	Não (>2×)
ML-DSA-44 (Dilithium2)	1.312	2.420	3.732	Não (>3×)
SLH-DSA-128f (SPHINCS+)	32	17.088	17.120	Não (>13×)

Table 1. Compatibilidade dos algoritmos PQC com o limite de 1.232 bytes por transação da Solana.

Conforme a Tabela 1, o Falcon-512 é o único algoritmo PQC padronizado pelo NIST que se torna compatível com o limite da Solana quando combinado com a estratégia

de armazenamento da chave pública em PDA, eliminando a necessidade de transmitir os 897 bytes a cada transação e resultando em 54% de ocupação do limite. Todos os demais candidatos excedem o limite mesmo com essa otimização: o ML-DSA-44 gera assinaturas de 2.420 bytes, e o SLH-DSA-128f produz assinaturas de 17.088 bytes. O Falcon-1024, embora mais seguro (nível 5 NIST), também é inviável com assinaturas de 1.280 bytes que por si só já excedem o limite.

O processo de assinatura do Falcon utiliza amostragem Gaussiana discreta sobre reticulados NTRU, uma operação que depende de aritmética de ponto flutuante de dupla precisão (IEEE 754) para garantir precisão numérica [20]. Esta dependência de ponto flutuante é uma característica intrínseca do projeto do Falcon, diferenciando-o de outros esquemas como o Dilithium (ML-DSA), que opera exclusivamente com aritmética inteira. Por outro lado, o Falcon oferece assinaturas significativamente menores: enquanto o Dilithium no nível de segurança 2 gera assinaturas de 2.420 bytes [2], o Falcon-512 produz assinaturas de aproximadamente 666 bytes, característica que o torna o candidato mais adequado para redes com restrições de tamanho de transação.

2.3. Trabalhos Relacionados

A integração de criptografia pós-quântica em redes blockchain tem sido objeto de investigação crescente, motivada pela ameaça representada por computadores quânticos às primitivas criptográficas clássicas. Yang et al. [13] apresentam um levantamento abrangente sobre o tema, categorizando as soluções em duas abordagens principais: blockchains pós-quânticas, que utilizam algoritmos criptográficos clássicos resistentes a ataques quânticos, e blockchains quânticas, que exploram o poder de computadores e redes quânticas (subdividindo-se em arquiteturas híbridas e sistemas totalmente quânticos). Neste contexto, os autores destacam que um dos principais desafios para a adoção da criptografia pós-quântica reside na dificuldade de alcançar alta eficiência, uma vez que algoritmos resistentes tendem a apresentar um aumento significativo no tamanho de chaves e assinaturas ou uma maior complexidade computacional, o que impacta o desempenho das redes

No ecossistema Ethereum, a LACChain desenvolveu um framework para verificação on-chain de assinaturas Falcon-512 utilizando contratos Solidity na rede Besu [15]. Embora funcional, essa abordagem revelou-se ineficiente devido aos altos custos de gas da verificação puramente em Solidity, motivando a proposição de soluções nativas. A proposta EIP-7619 [16] introduziu um contrato pré-compilado para verificação de assinaturas Falcon-512 no endereço `0x65`, aceitando chaves públicas de 897 bytes, assinaturas de até 666 bytes e mensagens de comprimento arbitrário. Posteriormente, a EIP-8052 [17] estendeu essa abordagem com três contratos pré-compilados distintos: `FALCON_CORE`, `FALCON_HASH_TO_POINT_KECCAKPRNG` e `FALCON_HASH_TO_POINT_SHAKE256`. Ambas as propostas adotam uma estratégia análoga ao syscall nativo proposto neste trabalho visto que executam a verificação criptográfica fora do ambiente restrito da máquina virtual, contornando as limitações da EVM.

Na rede Algorand, a adoção do Falcon é mais avançada em termos de implantação em produção. A plataforma implementou assinaturas Falcon-1024 para a geração de Provas de Estado pós-quânticos a cada 256 rounds de consenso [18]. Em novembro de 2025, a Algorand executou a primeira transação pós-quântica em mainnet utilizando assinatura lógicas (Logic Signatures) que encapsulam chaves públicas Falcon em contratos

inteligentes stateless. Contudo, a escolha do Falcon-1024 implica assinaturas de aproximadamente 1.538 bytes e chaves públicas de 1.793 bytes, dimensões incompatíveis com o limite de 1.232 bytes por transação imposto pela Solana [9]. Essa restrição justifica a adoção do Falcon-512 no presente trabalho, que oferece assinaturas de 666 bytes mantendo viabilidade dentro dos limites da rede.

Diferentemente dos trabalhos supracitados, a contribuição deste estudo reside na proposta e implementação concreta de uma modificação no runtime da Solana através de um syscall nativo (`sol_falcon512_verify`), acompanhada de um benchmark comparativo com 20.360 transações que quantifica o overhead da verificação PQC em termos de tempo de assinatura, tempo de confirmação, tamanho de transação e consumo de Compute Units. Enquanto as propostas do Ethereum (EIPs e LACChain) permanecem em estágio de draft ou limitadas pela ineficiência do Solidity, e a Algorand utiliza Falcon-1024 com parâmetros incompatíveis com a Solana. Dessa maneira, este trabalho oferece uma solução end-to-end validada experimentalmente que demonstra a viabilidade operacional da transição pós-quântica na Solana com o algoritmo Falcon-512.

3. Materiais e Métodos

O presente estudo adota uma abordagem experimental estruturada em duas fases complementares: (1) implementação de um programa híbrido como prova de conceito, e (2) modificação do runtime da Solana para suporte nativo à verificação PQC via syscall. O Falcon-512 foi selecionado por apresentar o menor tamanho combinado entre os algoritmos PQC, com assinaturas de 666 bytes e chaves públicas de 897 bytes, sendo o único candidato viável dentro do limite de 1.232 bytes por transação da Solana [9] quando a chave pública é armazenada em uma PDA.

3.1. Arquitetura Híbrida

O tamanho combinado dos dados Falcon-512 (897 bytes de chave pública + 666 bytes de assinatura + 32 bytes de hash = 1.595 bytes) excede o limite de 1.232 bytes por transação da Solana, inviabilizando o envio de todos os componentes em uma única instrução. A arquitetura proposta resolve esta restrição separando o fluxo em duas transações distintas, com responsabilidades divididas entre componentes off-chain e on-chain conforme a Figura 1.

3.1.1. Componente Off-Chain: Gestão de Chaves e Assinatura

O cliente opera fora da blockchain e é responsável por três operações: (i) geração do par de chaves Falcon-512 utilizando a biblioteca `pqcrypto-falcon`, mantendo a chave privada estritamente local; (ii) assinatura do hash SHA-256 da mensagem de transferência; e (iii) construção das transações que serão submetidas à rede.

O cliente gera, então, localmente um par de chaves Falcon-512 junto ao par Ed25519 nativo e registra a chave pública Falcon (897 bytes) em uma PDA derivada do endereço Ed25519 do usuário ($seeds = [\texttt{pqc_account}, pk_{ed25519}]$). Para cada transação subsequente, o cliente assina o hash SHA-256 da mensagem com a chave privada Falcon-512 off-chain e submete à rede com dupla autenticação, sendo Ed25519 (taxas e identificação) e Falcon-512 (no payload da instrução). On-chain, o programa recupera a chave pública da PDA e invoca o syscall nativo para verificação.

3.1.2. Componente On-Chain: Programa Verificador

O programa on-chain, compilado para bytecode SBF (Solana BPF), expõe três instruções identificadas por um byte de tag:

```
pub enum PqcInstructionTag {
   Initialize = 0, // Registrar chave publica PQC
   Verify = 1, // Verificar assinatura (sem transferencia)
   Transfer = 2, // Verificar + transferir lamports
}
```

Vinculação de Identidade via PDA. A instrução `Initialize` (tag=0) cria uma Program Derived Address (PDA) [21] que vincula a identidade Ed25519 do usuário à sua chave pública pós-quântica. A PDA é derivada deterministicamente a partir do endereço Ed25519 do proprietário:

```
fn find_pqc_account_pda(program_id: &Pubkey, owner: &Pubkey)
   -> (Pubkey, u8)
{
   Pubkey::find_program_address(
      &[b"pqc_account", owner.as_ref()], program_id
   )
}
```

A conta PDA armazena 946 bytes com o seguinte formato:

```
pub const PQC_ACCOUNT_SIZE: usize =
   8 + // discriminador
   32 + // owner (Chave publica Ed25519)
   FALCON512_PUBKEY_SIZE + // chave publica Falcon-512 (897 bytes)
   8 + // timestamp i64
   1; // is_initialized
```

A criação da conta utiliza `invoke_signed` com as seeds da PDA, garantindo que apenas o programa pode criar e controlar esta conta:

```
let create_ix = system_instruction::create_account(
   payer.key, pqc_account.key,
   rent.minimum_balance(PQC_ACCOUNT_SIZE),
   PQC_ACCOUNT_SIZE as u64, program_id,
);
invoke_signed(
   &create_ix,
   &[payer.clone(), pqc_account.clone(), sysprog.clone()],
   &[&[b"pqc_account", owner.as_ref(), &[bump]]],
)?;
```

Transferência Autorizada por Assinatura PQC. A instrução `Transfer` (tag=2) implementa o fluxo completo de autorização pós-quântica. O cliente constrói os dados da instrução com o formato `[tag (1B)] + [lamports (8B LE)] + [hash`

(32B)] + [sig_len (2B LE)] + [assinatura (sig_len B)]. O programa on-chain recupera a chave pública Falcon-512 armazenada na PDA e invoca o syscall nativo para verificação:

```
let (owner, pqc_pubkey, is_initialized) =
   unpack_owner_and_pubkey(&pqc_account.try_borrow_data()?)?;

let result = unsafe {
   sol_falcon512_verify(
      message_hash.as_ptr(),
      message_hash.len() as u64,
      signature.as_ptr(),
      pqc_pubkey.as_ptr(),
      sig_len as u64,
   )
};

if Falcon512Result::from(result) != Falcon512Result::Success {
   return Err(ProgramError::InvalidArgument);
}

**pqc_account.try_borrow_mut_lamports()? -= lamports;
**dest_account.try_borrow_mut_lamports()? += lamports;
```

A invocação do syscall utiliza compilação condicional: a declaração `extern "C"` com `#[cfg(target_os = "solana")]` permite que o programa chame a função nativa no ambiente SBF. A transferência de lamports é realizada por manipulação direta dos saldos (sem CPI para o `system_program`), possível porque a PDA pertence ao programa.

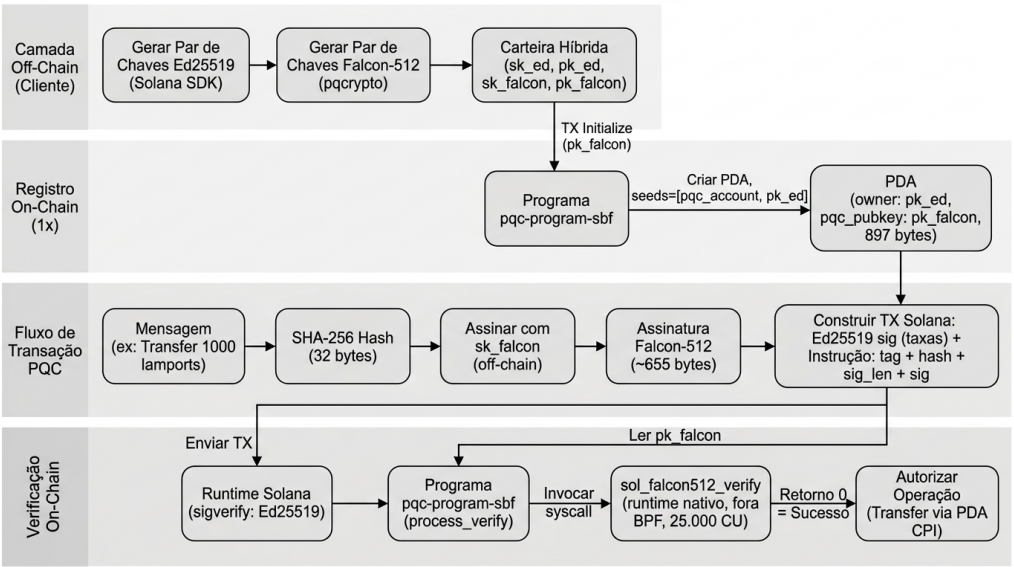

Figure 1. Fluxo completo da abordagem híbrida: geração de chaves, registro em PDA e verificação via syscall nativo.

A implementação da carteira híbrida na Devnet confirmou experimentalmente que a verificação criptográfica do Falcon-512 não pode ser executada dentro da SVM; a bib-

lioteca pqcrypto-falcon depende de aritmética de ponto flutuante [20], incompatível com o ambiente eBPF [12]. O armazenamento da chave pública na PDA e a construção de transações operaram corretamente; apenas a verificação criptográfica on-chain falhou, motivando a abordagem por syscall nativo.

3.2. Implementação do Syscall Nativo

O syscall sol_falcon512_verify foi implementado em um fork do repositório Agave[1] utilizando a macro declare_builtin_function, o mecanismo padrão do runtime da Solana para registrar funções nativas acessíveis a programas on-chain. A implementação segue três etapas: separação das informações da memória, validação estrutural e verificação criptográfica.

3.2.1. Mapeamento de Memória e Consumo de CU

A função recebe cinco parâmetros da VM (endereços e tamanhos da mensagem, assinatura e chave pública) e utiliza translate_slice para mapear endereços virtuais da SVM para a memória do processo do validador. O custo de 900 CU é consumido antes de qualquer operação:

```
declare_builtin_function!(
  SyscallFalcon512Verify,
  fn rust(
    invoke_context: &mut InvokeContext,
    message_addr: u64, message_len: u64,
    signature_addr: u64, pubkey_addr: u64,
    signature_len: u64,
    memory_mapping: &mut MemoryMapping,
  ) -> Result<u64, Error> {
    consume_compute_meter(invoke_context, FALCON512_VERIFY_COST)?;
    let check_aligned = invoke_context.get_check_aligned();

    let message = translate_slice::<u8>(
       memory_mapping, message_addr, message_len, check_aligned)?;
    let signature_bytes = translate_slice::<u8>(
       memory_mapping, signature_addr, sig_len as u64, check_aligned)?;
    let pubkey_bytes = translate_slice::<u8>(
       memory_mapping, pubkey_addr,
       FALCON512_PUBKEY_LEN as u64, check_aligned)?;
```

3.2.2. Validação e Verificação Criptográfica

Após o mapeamento de memória, a função valida as estruturas da chave pública e assinatura via from_bytes, que verifica a conformidade com as especificações do Falcon-512 (parâmetros do reticulado, formato interno). A verificação criptográfica final é delegada à biblioteca pqcrypto-falcon:

```
    let pubkey = match falcon512::PublicKey::from_bytes(pubkey_bytes) {
       Ok(pk) => pk,
```

[1] https://github.com/anza-xyz/agave

```
        Err(_) => return Ok(1), // chave publica invalida
    };
    let signature = match falcon512::DetachedSignature::from_bytes(
        signature_bytes
    ) {
        Ok(sig) => sig,
        Err(_) => return Ok(2), // assinatura invalida
    };
    match falcon512::verify_detached_signature(&signature, message, &
        pubkey) {
        Ok(()) => Ok(0), // verificacao bem-sucedida
        Err(_) => Ok(3), // verificacao criptografica falhou
    }
  }
);
```

O syscall retorna códigos padronizados: 0 (sucesso), 1 (chave pública inválida), 2 (assinatura inválida) e 3 (verificação falhou). A ativação ocorre via feature flag `enable_falcon512_syscall`, registrada nos módulos `feature-set` e `svm-feature-set` do Agave. O programa on-chain invoca o syscall através de uma declaração `extern "C"` com `#[cfg(target_os = "solana")]`, permitindo compilação condicional entre o ambiente SBF e testes locais.

O custo de 900 CU foi determinado utilizando a mesma metodologia empregada pela Solana para calibrar os custos das syscalls existentes. O comentário oficial no código-fonte do Poseidon (`execution_budget.rs`) documenta a razão de conversão **1 CU** $\approx$ **33 ns**. O procedimento de calibração consistiu em:

1. Executar 10.000 iterações de `falcon512::verify_detached_signature`.
2. Medir os tempos em nanosegundos e calcular percentis (Tabela 2).
3. Aplicar a fórmula $CU = t_{ns}/33$ e adicionar margem de 30% para acomodar variação entre hardware de validadores.

Métrica	**Tempo (ns)**	**CU (÷33)**
Média	17.123	519
Mediana	17.042	516
P95	18.708	567
P99	21.917	664
P99 + 30%	28.492	863
Valor adotado (arredondado)	—	**900**

Table 2. Calibração do custo do syscall sol_falcon512_verify (10.000 iterações, Mac Book Pro M3).

3.3. Metodologia de Benchmark

Os experimentos foram executados em um solana-test-validator modificado com o syscall PQC ativado, comparando duas abordagens de transferência de 1.000 lamports: (i) **Nativa (Ed25519)** VIA instrução `system_instruction::transfer` e (ii) **PQC (Falcon512)** por meio datransferência autorizada mediante verificação de assinatura Falcon-512 pelo syscall nativo `sol_falcon512_verify`. Avaliaram-se 20.360 transações (N

= 5, 25, 50, 100 e 10.000), medindo tempo de assinatura (μs), tempo de confirmação (ms), tamanho da transação (bytes) e Compute Units consumidas. É importante notar uma assimetria na contabilização de CU no fluxo Ed25519 nativo, a verificação criptográfica, que consome aproximadamente 2.280 CU internamente, ocorre no estágio de verificação de assinatura e não é registrada no processo de medição do Compute Unit, resultando em apenas 150 CU reportadas. Para possibilitar uma análise justa, além dos valores reportados pela medição do Compute Unit, a Tabela 4 discrimina o custo real da verificação criptográfica de cada abordagem.

4. Resultados

Os resultados experimentais confirmam a viabilidade operacional da transição pós quântica. A verificação do Falcon512 via syscall nativo mostrou ser computacionalmente mais eficiente que o padrão Ed25519, revelando que os principais trade-offs da adoção está concentrado no aumento do tamanho da transação, e não na complexidade criptográfica. Para validar essas premissas, o benchmark executou 20.360 transações distribuídas em cinco volumes (N = 5, 25, 50, 100 e 10.000), alcançando taxa de sucesso de 100% em todas as execuções. A Tabela 3 apresenta os resultados para os diferentes cenários avaliados.

N	Abordagem	Assin. (μs)	Conf. (ms)	TX (bytes)	CU
5	Ed25519 (nativa)	30,8	502,0	215	150
5	PQC Falcon-512	150,8	502,4	934	9.614
100	Ed25519 (nativa)	30,0	527,3	215	150
100	PQC Falcon-512	144,9	512,5	934	9.614
10.000	Ed25519 (nativa)	31,9	523,8	215	150
10.000	PQC Falcon-512	148,7	529,0	934	9.614

Table 3. Resultados do benchmark para diferentes volumes de transações. CU reportados pelo Compute Unit meter do runtime.

as métricas apresentam estabilidade ao longo dos diferentes volumes, confirmando a independência dos resultados em relação à carga. A análise detalhada a seguir utiliza os dados de N=10.000 como referência.

4.1. Tempo de Assinatura

A geração de assinaturas Falcon-512 apresentou tempo médio de 148,7 μs, aproximadamente 4,7$\times$ mais lenta que a Ed25519 (31,9 μs). Apesar do fator multiplicativo significativo, ambos os tempos permanecem na ordem de microssegundos. Considerando que o tempo de confirmação de uma transação é da ordem de 500 ms, o tempo de assinatura Falcon-512 representa menos de 0,03% da latência total. A estabilidade dos valores ao longo dos diferentes volumes (144,1–150,8 μs para Falcon-512 e 30,0–31,9 μs para Ed25519) indica que o overhead de assinatura é constante e independente da carga.

4.2. Tamanho da Transação

As transações PQC apresentaram tamanho médio de 934 bytes, representando um aumento de 4,3$\times$ em relação às transações Ed25519 nativas (215 bytes). Este aumento

decorre da inclusão da assinatura Falcon-512 (666 bytes), do hash SHA-256 (32 bytes) e dos metadados da instrução nos dados enviados ao programa. Apesar do aumento, o tamanho permanece dentro do limite de 1.232 bytes imposto pela Solana, com margem de 298 bytes disponíveis para metadados adicionais.

4.3. Tempo de Confirmação

Os tempos de confirmação mostraram-se equivalentes entre as abordagens, oscilando entre 502 e 532 ms. Para N=10.000, os valores foram de 523,8 ms (Ed25519) e 529,0 ms (Falcon-512), com diferença inferior a 1%. Contudo, esta equivalência é um artefato do ambiente de teste, o `solana-test-validator` opera como nó único com tempo de slot fixo, sem consenso distribuído (Tower BFT), propagação de blocos entre validadores ou priorização de transações por taxa. Consequentemente, a latência medida reflete essencialmente o intervalo entre slots, independentemente do tamanho ou complexidade da transação. Em um ambiente de produção com múltiplos validadores, o aumento de 4,3× no tamanho das transações PQC impactaria a latência de propagação e o throughput sob alta carga, aspectos que demandam investigação em ambiente multi-nó.

4.4. Compute Units

A análise revela que a verificação criptográfica do Falcon-512 via syscall nativo (900 CU) é aproximadamente 2,5× mais eficiente que a verificação Ed25519 (2.280 CU). O custo adicional observado no fluxo PQC (9.614 CUs total) decorre predominantemente do overhead do programa on-chain, leitura da PDA, deserialização da chave pública de 897 bytes, lógica de autorização e não da operação criptográfica em si.

Métrica	**Ed25519 Nativo**	**Falcon-512**	**Fator**
Compute Unit reportado	150	9.614	—
Verificação criptografica	2.280 (sigverify)	900 (syscall)	0,39×
Compute Unit real total estimado	2.430	9.614	3,96×
Tempo de assinatura	31,9 μs	148,7 μs	4,7×
Tamanho TX	215 bytes	934 bytes	4,3×
Tempo de confirmação	523,8 ms	529,0 ms	1,0×

Table 4. Comparação normalizada entre Ed25519 nativo e Falcon-512 (N=10.000), discriminando o custo real da verificação criptográfica.

5. Discussão

Os resultados demonstram que a integração de criptografia pós-quântica na Solana via syscall nativo é tecnicamente viável e computacionalmente menos onerosa do que uma comparação superficial sugeriria. A discrepância entre os 150 CU reportados pelo fluxo Ed25519 nativo e os 9.614 CU do fluxo Falcon512 decorre de uma assimetria estrutural na contabilização, a verificação Ed25519 (2.280 CU) é executada no estágio de *sigverify* dentro do validador, anterior à execução do programa e invisível ao processo de medição dos Compute Units, enquanto no fluxo PQC, diferentemente do Ed25519, toda a computação, inclusive a verificação criptográfica é contabilizada. Quando se consideram os custos reais totais (verificação + execução), o overhead do Falcon512 é de ∼3,96×, não 64×.

Mais relevante ainda, a operação criptográfica isolada do Falcon512 (900 CU) é aproximadamente 2,5× mais eficiente que a verificação Ed25519 (2.280 CU), evidenciando que o custo adicional origina-se do overhead do programa on-chain (leitura de PDA, parsing das contas e lógica de autorização) e não da primitiva pós-quântica.

É importante ressaltar uma limitação do ambiente experimental em relação aos tempos de confirmação equivalentes entre as abordagens (∼502–530 ms) são uma consequência do `solana-test-validator`, que opera como um nó único. Neste ambiente, não há consenso distribuído (Tower BFT), propagação de blocos entre validadores, priorização de transações por taxa. Consequentemente, a latência medida reflete essencialmente o intervalo entre slots, independentemente do tamanho ou da complexidade da transação. Em um ambiente de produção com múltiplos validadores, o aumento de 4,3× no tamanho das transações PQC (934 vs. 215 bytes) impactaria a latência de propagação e o throughput sob alta carga, aspectos que requerem investigação em ambiente multi-nó. As métricas que permanecem confiáveis neste setup experimental são o tempo de assinatura (operação local, independente da rede), o tamanho da transação (determinístico) e os Compute Units (medidos pelo runtime do validador, independentes da quantidade de nodos validadores funcionando).

6. Conclusão

Este trabalho demonstrou a viabilidade técnica da transição da rede Solana para criptografia pós-quântica através de um syscall nativo `sol_falcon512_verify`. A incompatibilidade do ambiente BPF com aritmética de ponto flutuante, requisito intrínseco ao Falcon-512 [20], foi contornada pela execução direta no runtime do validador [19]. Os 30.540 testes experimentais revelaram que a verificação criptográfica Falcon-512 (∼900 CUs via syscall isolado) é ∼2,5× mais eficiente que a verificação Ed25519 (2.280 CU no sigverify), e que o overhead total real do fluxo PQC é de ∼4× quando comparados os custos reais de ambas as abordagens (9.614 vs. 2.430 CUs). O custo adicional origina-se do overhead do programa on-chain, não da primitiva criptográfica. O principal trade-off residual da abordagem está no aumento do tamanho das transações (4,3×) e das chaves públicas armazenadas em PDA (28×), não em custo computacional. A implementação requer aprovação e coordenação da comunidade de validadores, mas os dados experimentais indicam que a integração de verificação pós-quântica ao runtime da Solana é operacionalmente viável.

References

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010, Page 37-38.

[2] D. J. Bernstein, J. Buchmann and E. Dahmen, *Post-Quantum Cryptography*, Springer, 1st Edition, 2009, Page 15.

[3] A. Narayanan, J. Bonneau, E. Felten, A. Miller and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*, Princeton University Press, 2016.

[4] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, *High-Speed High-Security Signatures*, Journal of Cryptographic Engineering, 2012.

[5] J.-P. Aumasson, *Serious Cryptography: A Practical Introduction to Modern Encryption*, No Starch Press, 2st Edition, 2025, Page 106.

[6] A. Yakovenko, "Solana: A new architecture for a high performance blockchain v0.8.13", 2018. https://solana.com/solana-whitepaper.pdf

[7] National Institute of Standards and Technology, "Post-Quantum Cryptography: FIPS Approved", 2024. https://csrc.nist.gov/news/2024/postquantum-cryptography-fips-approved

[8] Falcon Project, "Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU". https://falcon-sign.info/

[9] Solana Foundation, *Transactions: Instruction Format*, Solana Documentation. https://solana.com/pt/docs/core/transactions#instruction-format.

[10] P. W. Shor, *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*, SIAM Journal on Computing, 26(5), 1997, pp. 1484-1509.

[11] J. Proos and C. Zalka, *Shor's discrete logarithm quantum algorithm for elliptic curves*, Quantum Information and Computation, 3(4), 2003, pp. 317-344.

[12] R. Love, *Linux Kernel Development*, Addison-Wesley Professional, 3rd Edition, 2010, Chapter 2 (No Easy Use of Floating Point)

[13] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. Di Pietro and A. Erbad, *A Survey and Comparison of Post-quantum and Quantum Blockchains*, arXiv preprint arXiv:2409.01358, 2024.

[14] D. Hankerson, A. J. Menezes and S. Vanstone, *Guide to Elliptic Curve Cryptography*, Springer Science, 1st Edition, 2004, Chapter 4.

[15] LACChain, "On-Chain Verification of Post-Quantum Signatures", LACNet Documentation. https://lacnet.com/on-chain-verification-of-post-quantum-signatures

[16] M. Eum, J. Leal, and A. Velasco, "EIP-7619: Precompile Falcon512 Generic Verifier", Ethereum Improvement Proposals, 2024. https://eips.ethereum.org/EIPS/eip-7619

[17] M. Eum, J. Leal, and A. Velasco, "EIP-8052: Precompile for Falcon Support", Ethereum Improvement Proposals, 2025. https://eips.ethereum.org/EIPS/eip-8052

[18] Algorand Foundation, "Technical Brief: Quantum-Resistant Transactions on Algorand with Falcon Signatures", 2025. https://algorand.co/blog/technical-brief-quantum-resistant-transactions-on-algorand-with-fal

[19] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, Pearson, 4th Edition, 2015, Section 1.6 (System Calls).

[20] T. Pornin, "New Efficient, Constant-Time Implementations of Falcon", Technical report, 2019. https://falcon-sign.info/falcon-impl-20190918.pdf

[21] Solana Documentation, "Program Derived Addresses (PDAs)". https://solana.com/docs/core/pda